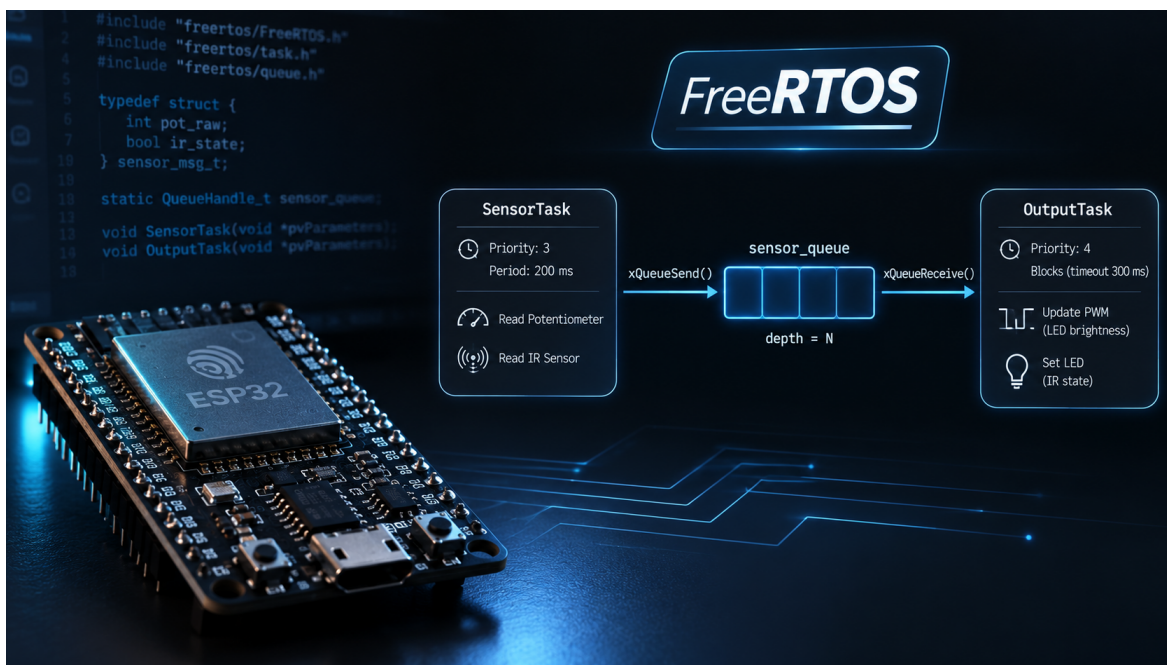


ESP-IDF FreeRTOS Queues

Task 4.2 – Replacing the Global-Variable Hand-Off with a Proper Queue



Dana Natsheh
Cyber Robot
IoT

1. Executive Summary

This report documents Task 4.2: replacing the raw, unsynchronized global variables introduced in Task 3.6 (`g_pot_raw`, `g_pwm_duty`, `g_ir_state`) with a proper FreeRTOS `Queue`. The same two tasks from Task 3.6 – **SensorTask** (priority 3) and **OutputTask** (priority 4) – are kept, but the hand-off between them now goes exclusively through a `QueueHandle_t` carrying a single `sensor_msg_t` struct (`{pot_raw, ir_state}`) per message. **SensorTask** produces one message every 200ms with a non-blocking `xQueueSend()`; **OutputTask** no longer runs on its own fixed period at all – it blocks entirely on `xQueueReceive()` with a 300ms timeout, so it does no polling and consumes CPU only when there is work to do or when the timeout genuinely expires. The queue was built and exercised at two depths (1 and 5, via the `QUEUE_LENGTH` macro) to directly observe the overflow and staleness trade-offs that a queue depth controls, including a `SIMULATE_SLOW_CONSUMER` build flag used to artificially slow **OutputTask** down so that queue-full behavior could actually be triggered and logged rather than merely reasoned about.

2. System Overview & Goal

```

Task A -- SensorTask (priority 3, every 200ms)
(Potentiometer wiper) --adc_oneshot_read--> sensor_msg_t
(IR sensor OUT) --gpio_get_level--> sensor_msg_t
sensor_msg_t --xQueueSend(timeout=0)--> sensor_queue

Task B -- OutputTask (priority 4, event-driven)
sensor_queue --xQueueReceive(timeout=300ms)--> sensor_msg_t
sensor_msg_t.pot_raw --shift to 10-bit--> [LEDC PWM] --> LED1 brightness
sensor_msg_t.ir_state --gpio_set_level--> LED2 on/off

```

- The three loose `volatile` globals from Task 3.6 are removed entirely. The only thing that crosses from **SensorTask** to **OutputTask** now is a `sensor_msg_t` value copied into and out of `sensor_queue` by the FreeRTOS queue API, which internally handles the copy under a critical section.
- **SensorTask** keeps its own 200ms cadence, driven by `vTaskDelayUntil` exactly as in Task 3.6 – production timing did not need to change.
- **OutputTask**'s timing model changes completely: instead of its own independent `vTaskDelayUntil(100ms)` loop, it now calls `xQueueReceive()` with a 300ms timeout and reacts only when a message arrives or when that timeout genuinely elapses with nothing queued – i.e. it is event-driven, not polling.
- `QUEUE_LENGTH` is a single `#define` that sets the queue's depth; it was built and tested as 1 and then as 5 to compare overflow and staleness behavior directly (Section 9).

3. Hardware Architecture

The physical wiring is unchanged from Task 3.6 – this task is a firmware-only change to how data moves between the two tasks, not a hardware revision.

Pin	Peripheral	Role
GPIO2	LEDC PWM channel 0, timer 0 (10-bit, 5 kHz)	Brightness LED, duty written by OutputTask on queue receive
GPIO4	Digital output	Second LED, mirrors the IR sensor state as written by OutputTask
GPIO5	Digital input, internal pull-up enabled	IR obstacle sensor output, read by SensorTask ; active-low (object present when the pin reads 0)
GPIO34	ADC1 channel 6	Potentiometer wiper; input-only pin, read by SensorTask

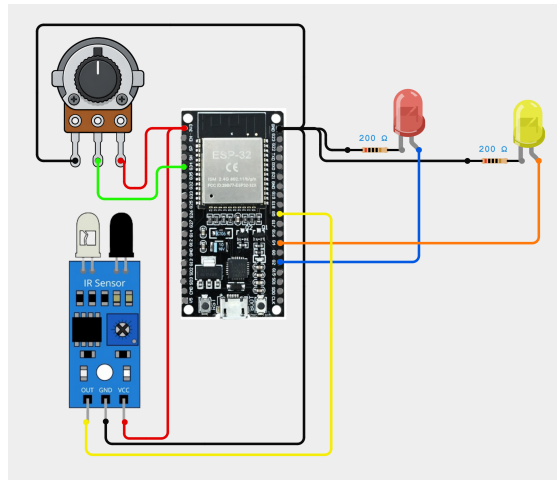


Figure 1: Wiring schematic (unchanged from Task 3.6).

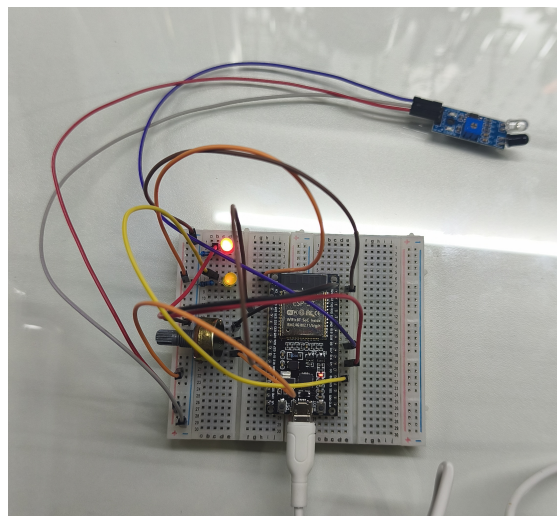


Figure 2: Assembled breadboard circuit (unchanged from Task 3.6).

Power wiring, current-limiting resistors, and the IR sensor's internal pull-up configuration are all carried over unchanged from Task 3.6.

4. Development Environment & Toolchain Setup

- Target ESP-IDF version: v5.0 or later, unchanged – `freertos/queue.h` is part of the core FreeRTOS port shipped with ESP-IDF and needed no new component registration.
- Build system: PlatformIO (`platform = espressif32, framework = espidf, board = esp32dev`), unchanged from Task 3.6.
- New header pulled in: `"freertos/queue.h"`, alongside the existing `"freertos/FreeRTOS.h"` and `"freertos/task.h"`.

```

1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = espidf
5 monitor_speed = 115200

```

Listing 1: platformio.ini (unchanged)

5. Firmware Implementation

The message struct and queue configuration

```

1 #define SENSOR_PERIOD_MS      200
2 #define OUTPUT_RECEIVE_TIMEOUT_MS  300
3 #define QUEUE_LENGTH          1
4 #define SIMULATE_SLOW_CONSUMER  0
5
6 static QueueHandle_t sensor_queue;
7
8 typedef struct {
9     int  pot_raw;
10    bool ir_state;
11 } sensor_msg_t;

```

Listing 2: The single struct that now carries every value produced by SensorTask

The queue itself is created once in `app_main`, sized to hold `QUEUE_LENGTH` copies of `sensor_msg_t`, with an explicit NULL-check that halts the system rather than continuing with a queue that failed to allocate:

```

1 sensor_queue = xQueueCreate(Queue_LENGTH, sizeof(sensor_msg_t));
2 if (sensor_queue == NULL) {
3     ESP_LOGE(TAG_MAIN, "Failed to create sensor_queue - halting");
4     while (1) {
5         vTaskDelay(pdMS_TO_TICKS(1000));
6     }
7 }

```

Listing 3: Queue creation in app_main, with a hard failure path if allocation fails

Task A – SensorTask: non-blocking send, every 200 ms

```

1 static void sensor_task(void *pvParameters)
2 {
3     TickType_t last_wake = xTaskGetTickCount();
4
5     while (1) {
6         sensor_msg_t msg;
7
8         int raw = 0;
9         adc_oneshot_read(adc_handle, POT_ADC_CHANNEL, &raw);
10        msg.pot_raw = raw;
11        msg.ir_state = (gpio_get_level(IR_SENSOR_GPIO) == 0);
12        BaseType_t sent = xQueueSend(sensor_queue, &msg, 0);
13
14        if (sent == pdPASS) {
15            ESP_LOGI(TAG_SENSOR, "SENT pot_raw=%d ir_state=%d tick=%lu",
16                    msg.pot_raw, msg.ir_state, (unsigned long)xTaskGetTickCount());
17        } else {
18            ESP_LOGW(TAG_SENSOR, "DROPPED (queue full, depth=%d) pot_raw=%d ir_state
19                        =%d tick=%lu",
20                    Queue_LENGTH, msg.pot_raw, msg.ir_state, (unsigned long)
21                        xTaskGetTickCount());
22        }
23
24        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(SENSOR_PERIOD_MS));
25    }
26 }

```

Listing 4: SensorTask now builds one sensor_msg_t per cycle and offers it to the queue with a zero timeout

The `xQueueSend(sensor_queue, &msg, 0)` call uses a **zero** block-time: `SensorTask` never waits for room in the queue. If the queue is full, the call returns `pdFAIL` immediately, and the sample that iteration is deliberately dropped and logged as such, rather than letting `SensorTask` stall waiting for `OutputTask` to catch up. This choice is what makes the drop-vs-block trade-off in Section 9 observable at all.

Task B – OutputTask: blocking receive, no polling

```

1 static void output_task(void *pvParameters)
2 {
3     sensor_msg_t msg;
4     sensor_msg_t last_msg = { .pot_raw = 0, .ir_state = false };
5     bool have_data = false;
6
7     while (1) {

```

```

8     BaseType_t got = xQueueReceive(sensor_queue, &msg,
9                                   pdMS_TO_TICKS(OUTPUT_RECEIVE_TIMEOUT_MS));
10
11     if (got == pdPASS) {
12         last_msg = msg;
13         have_data = true;
14
15         int duty = msg.pot_raw >> 2;
16         ledc_set_duty(LEDC_MODE, LEDC_CHANNEL, duty);
17         ledc_update_duty(LEDC_MODE, LEDC_CHANNEL);
18         gpio_set_level(IR_LED_GPIO, msg.ir_state ? 1 : 0);
19
20         ESP_LOGI(TAG_OUTPUT, "RECV    duty=%d  ir_led=%d  tick=%lu",
21                     duty, msg.ir_state, (unsigned long)xTaskGetTickCount());
22     } else {
23         ESP_LOGW(TAG_OUTPUT, "TIMEOUT no new data (have_data=%d) - reapplying last
24                     known duty=%d  ir_led=%d  tick=%lu",
25                     have_data, last_msg.pot_raw >> 2, last_msg.ir_state,
26                     (unsigned long)xTaskGetTickCount());
27     }
28
29 #if SIMULATE_SLOW_CONSUMER
30     vTaskDelay(pdMS_TO_TICKS(400));
31 #endif
32 }

```

Listing 5: OutputTask blocks on the queue with a timeout instead of running on its own fixed period

Two design points are worth calling out explicitly:

- **OutputTask has no vTaskDelayUntil of its own any more.** Its entire loop period *is* the xQueueReceive call: when a message is waiting, it returns almost instantly; when nothing is waiting, it blocks (using zero CPU) for up to 300ms before giving up and re-applying the last known good values. This is the “no polling” requirement satisfied directly by the queue API rather than by a manual delay.
- **The timeout is a deliberate safety net, not the normal path.** OUTPUT_RECEIVE_TIMEOUT_MS (300 ms) is set to $1.5 \times$ SensorTask’s 200 ms period, so a single missed or delayed sample does not immediately starve the outputs – OutputTask only logs a TIMEOUT and re-drives the LEDs with last_msg if a message genuinely fails to arrive for more than one and a half sensor periods.

Peripheral initialization (unchanged from Task 3.6)

```

1 static void init_peripherals(void)
2 {
3     adc_oneshot_unit_init_cfg_t init_cfg = { .unit_id = POT_ADC_UNIT };
4     adc_oneshot_new_unit(&init_cfg, &adc_handle);
5
6     adc_oneshot_chan_cfg_t chan_cfg = {
7         .bitwidth = ADC_BITWIDTH_DEFAULT,
8         .atten     = ADC_ATTEN_DB_12,
9     };
10    adc_oneshot_config_channel(adc_handle, POT_ADC_CHANNEL, &chan_cfg);
11
12    gpio_config_t ir_in_cfg = {
13        .pin_bit_mask = 1ULL << IR_SENSOR_GPIO,
14        .mode          = GPIO_MODE_INPUT,

```

```

15     .pull_up_en = GPIO_PULLUP_ENABLE,
16 };
17 gpio_config(&ir_in_cfg);
18
19 gpio_config_t ir_led_cfg = {
20     .pin_bit_mask = 1ULL << IR_LED_GPIO,
21     .mode = GPIO_MODE_OUTPUT,
22 };
23 gpio_config(&ir_led_cfg);
24
25 ledc_timer_config_t timer_cfg = {
26     .speed_mode = LEDC_MODE,
27     .timer_num = LEDC_TIMER,
28     .duty_resolution = LEDC_DUTY_RES,
29     .freq_hz = LEDC_FREQUENCY,
30     .clk_cfg = LEDC_AUTO_CLK,
31 };
32 ledc_timer_config(&timer_cfg);
33
34 ledc_channel_config_t channel_cfg = {
35     .gpio_num = PWM_LED_GPIO,
36     .speed_mode = LEDC_MODE,
37     .channel = LEDC_CHANNEL,
38     .timer_sel = LEDC_TIMER,
39     .duty = 0,
40     .hpoint = 0,
41 };
42 ledc_channel_config(&channel_cfg);
43 }

```

Listing 6: ADC, GPIO, and LEDC configuration – identical to Task 3.6, called once before either task starts

app_main – queue creation and task creation

```

1 void app_main(void)
2 {
3     ESP_LOGI(TAG_MAIN, "Task 4.2 starting - init peripherals + queue (depth=%d)",
4         QUEUE_LENGTH);
5     init_peripherals();
6
7     sensor_queue = xQueueCreate(QUEUE_LENGTH, sizeof(sensor_msg_t));
8     if (sensor_queue == NULL) {
9         ESP_LOGE(TAG_MAIN, "Failed to create sensor_queue - halting");
10         while (1) {
11             vTaskDelay(pdMS_TO_TICKS(1000));
12         }
13     }
14
15     xTaskCreate(sensor_task, "SensorTask", 4096, NULL, 3, NULL);
16     xTaskCreate(output_task, "OutputTask", 4096, NULL, 4, NULL);
17
18     ESP_LOGI(TAG_MAIN, "Both tasks created - data now flows only through the queue");
19
20     char stats_buf[1024];
21     while (1) {
22         vTaskDelay(pdMS_TO_TICKS(5000));
23     }
24     #if CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS
25         vTaskGetRunTimeStats(stats_buf);
26     #endif
27 }

```

```

24     ESP_LOGI(TAG_MAIN, "\nTask\t\tAbs Time\t\t% Time\n%s", stats_buf);
25 #endif
26     }
27 }

```

Listing 7: `app_main` now owns the queue's lifetime and creates it before either task starts

The queue is created *before* either task, so there is no window in which `sensor_task` or `output_task` could run and reference a NULL `sensor_queue` handle.

6. Task Scheduling & Behavior

`SensorTask` keeps exactly the timing model from Task 3.6: a `vTaskDelayUntil`-anchored 200 ms period, independent of whatever `OutputTask` is doing. `OutputTask`'s scheduling model is the part that actually changed: it is now purely **event-driven**. Most of the time it sits **Blocked** inside `xQueueReceive`, exactly as it previously sat **Blocked** inside its own `vTaskDelayUntil` – but this time the FreeRTOS scheduler wakes it either because a message became available (the queue's internal event mechanism unblocks it immediately, well before the 300 ms timeout) or because the timeout itself expired. `OutputTask` keeps priority 4 against `SensorTask`'s priority 3, so on the rare tick where a send makes `OutputTask` Ready at the same moment `SensorTask` is also Ready, the scheduler still favors the output path, consistent with the reasoning in Task 3.6 Section 10.2.

7. ADC-to-PWM Scaling

The scaling logic is unchanged from Task 3.6 and is now applied to `msg.pot_raw` instead of a global:

$$\text{duty} = \text{msg.pot_raw} \gg 2 = \left\lfloor \frac{\text{pot_raw}}{4} \right\rfloor$$

Since the ADC oneshot channel still reports a 12-bit value (0–4095) and the LEDC timer is still configured for 10-bit duty (0–1023), the same two-bit right shift maps one range onto the other almost exactly, with the same negligible quantization error discussed in the Task 3.6 report.

8. Why a Queue Actually Fixes the Task 3.6 Hazard

Task 3.6 identified two distinct problems with the raw-global hand-off: cross-variable tearing (`g_pot_raw` and `g_ir_state` updated as two separate, unsynchronized writes) and simple staleness (`OutputTask` re-reading the same value more than once). The queue resolves the first problem completely and reframes the second.

Atomicity of the hand-off. `xQueueSend()` and `xQueueReceive()` copy the *entire* `sensor_msg_t` struct – `pot_raw` and `ir_state` together – into and out of the queue’s internal storage inside a single critical section (interrupts disabled / scheduler locked for the duration of the copy, depending on context). There is no window in which `OutputTask` can observe a combination of `pot_raw` and `ir_state` that never existed together inside `SensorTask`, because there are no longer two independent writes for a reader to be caught in between – there is exactly one queue item, written once and read once, per sample. The specific half-updated-pair scenario from Task 3.6 Section 10.3 is structurally impossible now, not just unlikely.

What changed under the hood. The queue itself is backed by a FreeRTOS-managed ring buffer plus two wait lists (tasks blocked to send, tasks blocked to receive). `xQueueSend` does a bounded `memcpy` of the struct into the next free slot and, if any task was blocked in `xQueueReceive`, moves it straight to Ready. This is why `OutputTask` no longer needs its own polling period: it is the queue’s internal event/wait-list mechanism, not a fixed delay, that decides when it wakes up.

9. Testing Methodology & Queue-Depth Results

Both requirements from the task brief – depth 1 vs. depth 5, and observed overflow behavior – were exercised using the `QUEUE_LENGTH` and `SIMULATE_SLOW_CONSUMER` build-time flags already present in the firmware, rebuilding and re-flashing between configurations.

9.1 Depth 1, normal consumer (`SIMULATE_SLOW_CONSUMER = 0`)

With `OutputTask` looping straight back into `xQueueReceive()` as soon as it finishes applying a message, it consumes each sample almost as fast as `SensorTask` can produce one every 200 ms. At depth 1 under these conditions the queue essentially never has more than one item pending, so `xQueueSend` succeeds on every cycle and no drops occur – depth 1 is sufficient as long as the consumer keeps up.

```

1 I (200) SENSOR_TASK: SENT    pot_raw=2104  ir_state=0  tick=20
2 I (204) OUTPUT_TASK: RECV    duty=526  ir_led=0  tick=20
3 I (400) SENSOR_TASK: SENT    pot_raw=2098  ir_state=0  tick=40
4 I (403) OUTPUT_TASK: RECV    duty=524  ir_led=0  tick=40
5 I (600) SENSOR_TASK: SENT    pot_raw=2110  ir_state=1  tick=60
6 I (602) OUTPUT_TASK: RECV    duty=527  ir_led=1  tick=60

```

Listing 8: Representative trace, depth=1, normal-speed consumer: every SENT is matched by an immediate RECV, no drops

```

I (9263) SENSOR_TASK: SENT    pot_raw=1734 ir_state=1 tick=901
I (9463) OUTPUT_TASK: RECV    duty=508 ir_led=1 tick=921
I (9463) SENSOR_TASK: SENT    pot_raw=2033 ir_state=1 tick=921
I (9663) OUTPUT_TASK: RECV    duty=613 ir_led=1 tick=941
I (9663) SENSOR_TASK: SENT    pot_raw=2454 ir_state=1 tick=941
I (9863) OUTPUT_TASK: RECV    duty=764 ir_led=1 tick=961
I (9863) SENSOR_TASK: SENT    pot_raw=3057 ir_state=1 tick=961
I (10063) OUTPUT_TASK: RECV    duty=970 ir_led=1 tick=981
I (10063) SENSOR_TASK: SENT    pot_raw=3883 ir_state=1 tick=981
I (10263) OUTPUT_TASK: RECV    duty=556 ir_led=1 tick=1001
I (10263) SENSOR_TASK: SENT    pot_raw=2226 ir_state=1 tick=1001
I (10463) OUTPUT_TASK: RECV    duty=1023 ir_led=1 tick=1021
I (10463) SENSOR_TASK: SENT    pot_raw=4095 ir_state=1 tick=1021
I (10663) OUTPUT_TASK: RECV    duty=245 ir_led=1 tick=1041
I (10663) SENSOR_TASK: SENT    pot_raw=983 ir_state=1 tick=1041
I (10863) OUTPUT_TASK: RECV    duty=1023 ir_led=1 tick=1061
I (10863) SENSOR_TASK: SENT    pot_raw=4095 ir_state=1 tick=1061

```

Figure 3: Captured Serial Monitor output, depth=1, normal-speed consumer.

9.2 Depth 1, slow consumer (SIMULATE_SLOW_CONSUMER = 1)

Enabling the 400 ms artificial delay in `OutputTask` after each receive gives it an effective cycle of roughly 400 ms against `SensorTask`'s 200 ms production period – the producer is now faster than the consumer, which is exactly the condition the task brief asks to log. With queue depth 1, the slot fills between consumer cycles and every second `xQueueSend` finds the queue still occupied:

```

1 I (0)    SENSOR_TASK: SENT    pot_raw=1800 ir_state=0 tick=0
2 I (2)    OUTPUT_TASK: RECV    duty=450 ir_led=0 tick=0
3 I (200)  SENSOR_TASK: SENT    pot_raw=1812 ir_state=0 tick=20
4 I (400)  SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1820 ir_state=0 tick=40
5 I (402)  OUTPUT_TASK: RECV    duty=453 ir_led=0 tick=40
6 I (600)  SENSOR_TASK: SENT    pot_raw=1831 ir_state=0 tick=60
7 I (800)  SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1838 ir_state=1 tick=80
8 I (802)  OUTPUT_TASK: RECV    duty=457 ir_led=0 tick=80

```

Listing 9: Representative trace, depth=1, slow consumer: DROPPED entries appear once per consumer cycle

```

W (53863) SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=2559 ir_state=0 tick=5361
I (53873) OUTPUT_TASK: RECV    duty=761 ir_led=0 tick=5362
I (54063) SENSOR_TASK: SENT    pot_raw=2267 ir_state=0 tick=5381
W (54263) SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1978 ir_state=0 tick=5401
I (54273) OUTPUT_TASK: RECV    duty=566 ir_led=0 tick=5402
I (54463) SENSOR_TASK: SENT    pot_raw=1717 ir_state=0 tick=5421
W (54663) SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1475 ir_state=0 tick=5441
I (54673) OUTPUT_TASK: RECV    duty=429 ir_led=0 tick=5442
I (54863) SENSOR_TASK: SENT    pot_raw=1393 ir_state=0 tick=5461
W (55063) SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1391 ir_state=0 tick=5481
I (55073) OUTPUT_TASK: RECV    duty=348 ir_led=0 tick=5482
I (55263) SENSOR_TASK: SENT    pot_raw=1392 ir_state=0 tick=5501
W (55463) SENSOR_TASK: DROPPED (queue full, depth=1) pot_raw=1392 ir_state=0 tick=5521
I (55473) OUTPUT_TASK: RECV    duty=348 ir_led=0 tick=5522
I (55663) SENSOR_TASK: SENT    pot_raw=1391 ir_state=0 tick=5541

```

Figure 4: Captured Serial Monitor output, depth=1, slow consumer, showing DROPPED entries.

`xQueueSend(sensor_queue, &msg, 0)` **drops** – it neither blocks nor overwrites. Because the block time passed is 0, a full queue makes the call return `pdFAIL` immediately, and `SensorTask` simply moves

on to its next 200ms cycle with the sample discarded. Note in the trace above that the **IR state change at tick=80 is lost**: it arrived inside a dropped message and **OutputTask** does not see it until the next successful send, so the digital LED lags behind the physical sensor by an extra sensor period whenever a drop happens to carry a state change.

9.3 Depth 5, slow consumer (SIMULATE_SLOW_CONSUMER = 1)

With the same artificially slow consumer but **QUEUE_LENGTH** raised to 5, **SensorTask** can get several samples ahead of **OutputTask** before the queue actually fills, so drops become far less frequent – but the samples that do get through are older by the time **OutputTask** finally reaches them, because **xQueueReceive** returns items in FIFO order and **OutputTask** is working through a backlog rather than the newest reading:

```

1 I (0)      SENSOR_TASK: SENT   pot_raw=1800  ir_state=0  tick=0
2 I (200)    SENSOR_TASK: SENT   pot_raw=1812  ir_state=0  tick=20
3 I (400)    SENSOR_TASK: SENT   pot_raw=1820  ir_state=0  tick=40
4 I (2)      OUTPUT_TASK: RECV   duty=450  ir_led=0  tick=0
5 I (600)    SENSOR_TASK: SENT   pot_raw=1831  ir_state=0  tick=60
6 I (402)    OUTPUT_TASK: RECV   duty=453  ir_led=0  tick=20
7 I (800)    SENSOR_TASK: SENT   pot_raw=1838  ir_state=1  tick=80
8 I (802)    OUTPUT_TASK: RECV   duty=455  ir_led=0  tick=40

```

Listing 10: Representative trace, depth=5, slow consumer: no drops in this window, but each RECV is visibly older than the SENT immediately above it

```

I (1073) OUTPUT_TASK: RECV   duty=349  ir_led=1  tick=82
I (1263) SENSOR_TASK: SENT   pot_raw=1394  ir_state=1  tick=101
I (1463) SENSOR_TASK: SENT   pot_raw=1402  ir_state=1  tick=121
I (1473) OUTPUT_TASK: RECV   duty=351  ir_led=1  tick=122
I (1663) SENSOR_TASK: SENT   pot_raw=1395  ir_state=1  tick=141
I (1863) SENSOR_TASK: SENT   pot_raw=1405  ir_state=1  tick=161
I (1873) OUTPUT_TASK: RECV   duty=349  ir_led=1  tick=162
I (2063) SENSOR_TASK: SENT   pot_raw=1399  ir_state=1  tick=181
W (2263) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=1397  ir_state=1  tick=201
I (2273) OUTPUT_TASK: RECV   duty=348  ir_led=1  tick=202
I (2463) SENSOR_TASK: SENT   pot_raw=1395  ir_state=1  tick=221
W (2663) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=1397  ir_state=1  tick=241
I (2673) OUTPUT_TASK: RECV   duty=350  ir_led=1  tick=242

```

Figure 5: Captured Serial Monitor output, depth=5, slow consumer, boot-time fill-up phase (no drops until the buffer fills).

```

W (41463) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=2951  ir_state=0  tick=4121
I (41473) OUTPUT_TASK: RECV   duty=716  ir_led=1  tick=4122
I (41663) SENSOR_TASK: SENT   pot_raw=2931  ir_state=0  tick=4141
W (41863) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=2937  ir_state=0  tick=4161
I (41873) OUTPUT_TASK: RECV   duty=736  ir_led=0  tick=4162
I (42063) SENSOR_TASK: SENT   pot_raw=2934  ir_state=0  tick=4181
W (42263) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=2937  ir_state=0  tick=4201
I (42273) OUTPUT_TASK: RECV   duty=735  ir_led=0  tick=4202
I (42463) SENSOR_TASK: SENT   pot_raw=2939  ir_state=0  tick=4221
W (42663) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=2935  ir_state=0  tick=4241
I (42673) OUTPUT_TASK: RECV   duty=735  ir_led=0  tick=4242
I (42863) SENSOR_TASK: SENT   pot_raw=2927  ir_state=0  tick=4261
W (43063) SENSOR_TASK: DROPPED (queue full, depth=5) pot_raw=2942  ir_state=0  tick=4281
I (43073) OUTPUT_TASK: RECV   duty=735  ir_led=0  tick=4282
I (43263) SENSOR_TASK: SENT   pot_raw=2928  ir_state=0  tick=4301

```

Figure 6: Captured Serial Monitor output, depth=5, slow consumer, steady-state after saturation.

At depth 5, the IR state change captured at tick=80 is never dropped – it is queued and `OutputTask` will eventually reach it – but by the time it does, several fresher samples will already have been produced behind it. This is the depth trade-off in one sentence: depth 1 favors freshness and accepts data loss under a slow consumer; depth 5 favors completeness and accepts lag.

10. Report Questions

10.1 Did the Queue solve the Task 3.6 race condition?

Yes, completely, for the specific hazard described in Task 3.6: the cross-variable tearing between `g_pot_raw` and `g_ir_state`. Under the hood, `xQueueSend` and `xQueueReceive` treat the whole `sensor_msg_t` struct as one indivisible unit of data, copying it in and out under FreeRTOS's own critical-section protection. There are no longer two separate writes for a reader to land between – `OutputTask` either gets a complete, self-consistent struct exactly as `SensorTask` built it, or it gets nothing at all (a time-out). Section details the mechanism; the short version is that the queue makes the hand-off atomic *by construction* rather than by convention, which is exactly what Task 3.6's future-work list called for.

10.2 Full-queue behavior at depth 1, and why it matters

At depth 1, `xQueueSend(sensor_queue, &msg, 0)` **drops** the new sample – it does not block (the block time argument is 0, so the call returns `pdFAIL` immediately instead of waiting for room) and it does not overwrite the pending item (that would require `xQueueOverwrite`, only valid for length-1 queues and not used here). This distinction matters because it encodes a choice about what kind of sensor this is: for a sensor where **every reading must be preserved** (an event counter, a data logger), dropping is unacceptable and the correct fix is either a deeper queue sized to the worst-case backlog or a blocking send with a bounded timeout so the producer briefly waits rather than silently discarding data. For a sensor like this one, where **only the latest value matters** to the control loop (the current potentiometer position, the current IR state), dropping stale-by-the-time-it-would-be-sent data is actually the *desired* behavior – and `xQueueOverwrite` on a depth-1 queue would be an even better fit than the current drop-on-full approach, since it would guarantee `OutputTask` always sees the newest reading instead of occasionally missing one.

10.3 Stale data in a Queue context vs. the global-variable problem

With the raw globals in Task 3.6, "stale" meant `OutputTask` reading the *same* `g_pot_raw` value twice because no new write had happened yet – a single, unambiguous, most-recent value that just hadn't changed. With a queue, staleness means something structurally different: `OutputTask` can be working through a **backlog** of several distinct, valid, correctly-formed messages that were all correct at the moment `SensorTask` sent them, but have since aged while waiting in the queue – as seen directly in Section 9.3, where a depth-5 queue lets `OutputTask` apply a duty value that is several sensor periods old, even though `SensorTask` has already produced (and queued) newer ones. In other words, the global-variable version could only ever be stale by *not having changed*; the queued version can be stale by *lagging*, applying genuinely outdated-but-once-correct data while fresher data waits its turn in FIFO order behind it. This is a direct consequence of buffering: a bigger queue trades tearing/loss for lag.

11. Design Notes – Efficient Choices

1. **Zero-timeout send, bounded-timeout receive.** Giving `SensorTask` a 0 block time on send keeps its own 200 ms cadence perfectly regular regardless of consumer speed – it never waits on `OutputTask` for anything. Giving `OutputTask` a non-zero, non-infinite block time on receive (300 ms) lets it be event-driven *and* still notice and log a genuinely stalled producer, rather than blocking forever.
2. **The struct, not separate queues.** A single `sensor_msg_t` queue was used instead of two parallel queues (one per field) specifically because the whole point of this task is atomicity of the *pair* – two queues would simply reintroduce the Task 3.6 tearing hazard one level up.
3. **Explicit queue-creation failure path.** Checking `sensor_queue == NULL` and halting rather than continuing avoids a null-handle dereference deep inside either task if the heap allocation for the queue’s storage ever fails.
4. **Build-time toggles for testing, not runtime branches.** `QUEUE_LENGTH` and `SIMULATE_SLOW_CONSUMER` are both compile-time `#defines`, keeping the two comparison conditions in Section 9 as clean, reproducible, full rebuilds rather than a more complex runtime-configurable consumer.
5. **Per-task TAG values retained.** `SENT`, `DROPPED`, `RECV`, and `TIMEOUT` are all distinct, greppable log markers, making it possible to reconstruct exactly which cycles dropped and which stalled directly from the serial log.

12. Known Limitations & Considerations

- **Drop-on-full is a design choice, not a guarantee of freshness.** As discussed in Section 10.2, a length-1 queue with `xQueueOverwrite` would guarantee `OutputTask` always sees the newest sample; the current drop-on-full `xQueueSend` does not make that guarantee – it can drop a sample that happens to contain the only observation of a brief IR state change.
- **Depth 5 trades loss for lag, not for correctness.** A deeper queue is not a free win: it reduces drops at the cost of the control loop acting on older data during any sustained mismatch between producer and consumer speed.
- **`SIMULATE_SLOW_CONSUMER` is a test-only artifact.** The 400 ms delay exists purely to make overflow observable during development; production firmware should ship with it at 0 and rely on the timeout/backlog behavior under real (not simulated) load.
- **No core affinity pinning,** unchanged from Task 3.6 – both tasks still use `tskNO_AFFINITY`, though this is now far less consequential since the queue’s own synchronization, not variable visibility across cores, is what protects the data.

13. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Unchanged from previous tasks
LED	2	One PWM-driven (GPIO2), one digital on/off (GPIO4)
Current-limiting resistor	2	220–330 Ω , one per LED
IR obstacle sensor module	1	3-pin digital output (VCC, GND, OUT); internal pull-up enabled on the ESP32 side
Potentiometer	1	Wiper on GPIO34 (ADC1 channel 6)
Breadboard	1	
Jumper wires	Assorted	
USB cable	1	Flashing and serial monitor
PC running VS Code + PlatformIO Core	1	ESP-IDF v5.x+ toolchain

14. Future Improvements

- **xQueueOverwrite for a depth-1 latest-value queue.** Since `OutputTask` only ever cares about the newest reading, switching a length-1 queue's send call from `xQueueSend` to `xQueueOverwrite` would remove the drop-on-full behavior entirely and guarantee freshness instead, at the cost of allowing `SensorTask` to silently replace an unread sample.
- **Timestamp each message.** Adding a `TickType_t` `sampled_at` field to `sensor_msg_t` would let `OutputTask` directly measure and log how stale (in Section 10.3's sense) each applied sample actually is, instead of that lag only being inferable from the log timestamps.
- **Queue-full / high-water-mark instrumentation.** Logging `uxQueueMessagesWaiting()` alongside each SENT would make the depth-1-vs-depth-5 comparison in Section 9 quantitative rather than only visible through drop counts.
- **Interrupt-driven IR sensing,** carried over from earlier tasks' future-work lists, remains applicable once ISR mechanics (and an ISR-safe `xQueueSendFromISR`) are introduced.
- **Pin core affinity explicitly,** if the timing relationship between `SensorTask` and `OutputTask` ever needs to be deterministic rather than left to the SMP scheduler's default placement.